# Prompt Engineering

Getting better results from large language models

# What We'll Cover

## General Best Practices

- How to think about Copilot when prompting
- Be clear and direct
- "Don't do this" prompts
- Add context to your prompt
- Meta prompting

## Techniques

- Assuming a role
- Few-shot learning
- Chain of thought
- XML tagging
- DO and DON'T

# General Best Practices

# How to Think About Copilot When Prompting

> Think of it like a brilliant new hire — smart, keen, but missing your context.

The more precisely you explain what you want, the better the result.

**Golden rule:** Show your prompt to a colleague. If they'd be confused, the model will be too.

| Concept | Think of it like |
| --- | --- |
| **Copilot** | Your IDE — gets better as you learn it |
| **Prompt engineering** | A programming language — precision determines output |

# Be Clear and Direct

Explicit beats implicit. Don’t leave the model to guess.

### Less effective:

```
Create an analytics dashboard
```

### More effective:

```
Create an analytics dashboard. Include as many relevant features
and interactions as possible. Go beyond the basics to create
a fully-featured implementation.
```

> "Above and beyond" behaviour must be explicitly requested

# "Don’t do this" Prompts

Telling a model what *not* to do is less effective than telling it what *to* do.

### Less effective:

```
Do not use markdown
```

### More effective:

```
Write responses in smoothly flowing prose paragraphs.
```

### Less effective:

```
NEVER use ellipses
```

### More effective:

```
Your response will be read aloud by a TTS engine — avoid ellipses
as the engine won't know how to pronounce them.
```

> Redirect, don’t just restrict

# Add Context to Your Prompt

Explaining *why* helps the model understand your goals — and generalise from them.

**Without context:**

```
Use snake_case for all variable names.
```

**With context:**

```
This codebase follows PEP 8 conventions for a Python project.
Use snake_case for all variable names to stay consistent with
the existing style and pass our linter checks.
```

**Add context about:**

- The project or codebase
- The intended audience or consumer
- Constraints or non-functional requirements
- Why a specific approach is required

> Modern LLMs are smart enough to generalise from a good explanation

# Meta Prompting

Feed your prompt *back into the AI* and ask it to improve itself.

```
That was a high quality response, thanks! It seemed to take a while though.
Is there a way to clarify your instructions so you can get to a response
as good as this faster next time?

Read through your instructions and look for anything that made you take
longer than needed. Write targeted additions or changes to make a
request like this faster next time with the same quality.
```

**Common failure modes meta prompting can fix:**

- Slow starts
- Log-style status updates
- Repetitive fillers like *"Good catch"*, *"Aha"*, *"Got it—"*

> More complex models do better at this – try Claude Sonnet or GPT-4.1

# Techniques

# Assuming a Role

Setting a role focuses the model's expertise, tone, and scope.

```
You are an expert Java software engineer that always follows best practices
for the language and any libraries you use. You always write high quality,
simple, testable, and loosely coupled code.

Can you create a template project for a Spring Web API?
```

**Use roles to:**

- Set expertise – `"You are a cybersecurity expert"`
- Define tone – `"You are a friendly teacher for beginners"`
- Restrict scope – `"You are a SQL assistant. Only answer database questions."`

**When to use:** Creating agent files, focusing on a specific task, keeping responses concise

# Few-Shot Learning

Include a small number of input/output examples to steer the model toward the format and structure you want.

```
Input:
<review id="1">
    I love this product, highly recommend you buy it!
</review>

Output:
<assistant_response id="1">
    Positive
</assistant_response>

Input:
<review id="2">
    Terrible quality, broke after one day.
</review>

Output:
```

**Tips:** Use 3–5 examples. Mark them clearly, cover edge cases.

**When to use:** When you need rigid, specifically formatted output

# Chain of Thought

Ask the model to reason *before* answering — reduces errors on complex tasks.

**Simple version:**

```
Think through this step by step before giving your final answer.
```

**More explicit:**

```
Before answering, write out your reasoning in <thinking> tags,
then provide your final answer in <answer> tags.
```

**Or ask clarifying questions first:**

```
Do not make assumptions. Ask me any refining questions you need
before proceeding.
```

**When to use:** Complex or multi-step tasks, maths, logic, debugging, anything where the model is missing edge cases

# XML Tagging

Structure complex prompts with XML tags to clearly separate intent, data, and constraints.

```
You are an expert at reading financial statements.

<instructions>
  Create a summary of all the expenses and income
  this company has had in the last 2 years.
</instructions>

<data>
  {{financial_data}}
</data>

<tone>
  Use a professional tone. Get straight to the point.
</tone>
```

Use consistent, descriptive tag names. Nest tags where there is a natural hierarchy.

**When to use:** Complex prompts mixing instructions, data, and formatting requirements

# DO and DON’T

Giving the model an explicit list of DOs and DON’Ts guides behaviour clearly and reliably.

```
- DO: Read all relevant instruction files before starting
- DO: Write unit tests to cover any code you produce
- DO: Follow the existing code style and naming conventions

- DON'T: Install any new libraries — use what's already in the project
- DON'T: Modify files outside the scope of the ticket
- DON'T: Leave TODO comments — either implement it or raise an issue
```

Simple, but effective.

**When to use:** Guiding the model away from or toward specific actions — especially in agent files and `copilot-instructions.md`

# Combining Techniques

Techniques work best in combination — don't treat them as mutually exclusive.

| Technique | Best paired with |
| --- | --- |
| **Role** | DO/DON'T — define persona + guardrails |
| **Few-shot examples** | XML tags — structure examples clearly |
| **Chain of thought** | Role — *"You are a senior engineer. Think step by step."* |
| **Meta prompting** | Any — improve any prompt that produces mediocre results |
| **Context** | Everything — always explain the *why* |

> Plan your prompt before you write it — list the required actions first

# Summary

| Technique | When to use |
| --- | --- |
| **Be clear and direct** | Always – the most fundamental principle |
| **Add context** | When instructions might seem arbitrary |
| **Few-shot examples** | When format or structure is hard to describe |
| **XML tags** | Complex prompts mixing instructions and data |
| **Role** | Expertise, tone, or scope focus |
| **Chain of thought** | Multi-step reasoning or complex tasks |
| **DO / DON'T** | Explicit behavioural guardrails |
| **Meta prompting** | Improving prompts that give mediocre results |

# Questions?

**Resources:**

- Anthropic Prompt Engineering Guide
- OpenAI Prompt Engineering Guide
- GitHub Copilot Best Practices